Compiler Design Lab 5

FIRST Set Computation (No Epsilon)

Objective

In this lab, you will write a Python program to compute the **FIRST set** for each nonterminal in a context-free grammar.

This task covers the simplified case:

- Nonterminals → single uppercase letters (`A`, `B`, `S`)
- Terminals → single lowercase letters (`a`, `b`, `c`)
- No epsilon productions
- No OR (`|`) symbol
- Each production is written separately

Background

The **FIRST(A)** of a nonterminal `A` is the set of terminals that can appear as the **first symbol** in any string derived from `A`.

Example

If we have:

S → AB
A → a
B → b

Then:

FIRST(A) = {a}
FIRST(B) = {b}
FIRST(S) = {a}

Because `S → AB`, and `A → a`, so any string derived from `S` must start with `a`.

Algorithm Rules (No Epsilon Case)

Since we are not handling epsilon in this task, the rules are simplified:

Rule 1

If:

A → a...

(where a is a terminal)

Add a to FIRST(A)

Rule 2

If:

A → B...

(where B is a nonterminal)

Add everything in FIRST(B) to FIRST(A)

Important:

You must repeat the process until no new elements are added to any FIRST set.

This is called a **fixed-point iteration**.

Implementation Requirements

Your program must:

1. Store grammar in a dictionary
2. Use a dictionary of sets to store FIRST sets
3. Use loops (no recursion)
4. Work for any grammar that follows the lab restrictions
5. Not hardcode answers
